Simulate the fleet forward. Choose the best decision sequence before committing the next action.

A supervisory layer that forecasts fleet conditions, evaluates multi-step control sequences under hard constraints, and returns the next policy to the existing control plane.

Aurelius sits above the scheduler. It does not replace Kubernetes, Slurm, or a serving router, and it never touches payloads. At each control period, Aurelius reads the current fleet state and generates candidate multi-step policies. It simulates each policy across the planning horizon, rejects candidates that violate SLA, capacity, power, or operator constraints, and returns the first action from the highest-scoring feasible sequence.

+724% average SLA-safe goodput per dollar

MEAN OF +698% / +718% / +755% (PJM / ERCOT / CAISO), AN 8.24× GP/$ RATIO VS A RECONSTRUCTED PRODUCTION-CLASS BASELINE · UNCAPPED REPLAY OF PUBLIC PRODUCTION TRACES (1.55M REQUESTS) · BETTER SLA AT ~84% FEWER GPU-HOURS

The result is driven by two effects. First, the fixed-policy baseline loses efficiency as the uncapped replay becomes saturated, while Aurelius re-evaluates its policy at each control period. Second, the ablation shows that the largest additional gain appears when the planner can evaluate valid policy combinations outside the original predefined set. The simulator, objective, cost model, constraints, and replay inputs remain fixed across every arm.

UNCAPPED HIGH-LOAD REPLAY · SIMULATED, EVIDENCE NOT A GUARANTEE · READ-ONLY, METADATA ONLY, NO PAYLOADS

00 Abstract

Conventional infrastructure stacks, including the unified control planes that consolidate a fleet into one view, optimize the next decision: the next placement, the next scale action, the next route, each chosen locally against the state visible now. Aurelius changes what gets optimized. It treats the fleet as a system with a trajectory, forecasts that trajectory forward, and optimizes the path rather than the step.

Aurelius is a supervisory control layer that operates above the existing scheduler. Each control period it generates candidate multi-step policies over the controls the stack exposes, simulates them under hard constraints, and returns the first action of the highest-scoring feasible sequence.

In an uncapped replay of three selected public production-trace windows totaling **1.55 million requests** (the PJM, ERCOT, and CAISO expensive-power windows), Aurelius reaches an **8.24× mean**

gp/$ ratio against a reconstructed production-class baseline, where **gp/$** is SLA-safe goodput, SLA-compliant served tokens, per infrastructure dollar. That is a **+724% average** gain (per market +698% / +718% / +755%), while holding SLA strictly better and using **~84% fewer GPU-hours**. Aurelius also lowers GPU-hours and SLA-violation rates, so the gain is not produced by increasing spend or relaxing SLA. The magnitude is load-dependent and requires operator-data validation.

Sections 05–07 separate the operating conditions that produce the magnitude from the architectural mechanism measured by the ablation. Because the model, objective, cost assumptions, and gates were fixed, the difference is attributable to the policies the planner could represent and evaluate.

All reported results are deterministic simulated replays of public production traces. The ratio is load-dependent and applies only to the selected uncapped windows (§07). Results require live-telemetry calibration before any production claim, and are not a deployment. Aurelius runs read-only and needs only scheduler metadata, never payloads or model weights.

01 From one-step control to multi-period planning

Most schedulers, and the control planes above them, optimize a current-state or local objective: availability, fairness, utilization, latency, queue state, immediate capacity, immediate price. Each is reasonable in isolation. But the economic outcome of a decision is set by constraints that arrive **after** it is made: power and capacity headroom, congestion, memory and topology pressure, demand, and the price of power, which on wholesale markets can move several-fold within a day.

Unified telemetry improves current-state decisions, but it does not account for constraints that arise later in the planning horizon. A current-state control plane can choose an action based on the conditions visible now. It cannot compare how alternative action sequences evolve under forecasted demand, capacity, topology, and power conditions. That distinction matters because placement, capacity, and routing decisions create downstream costs that may be expensive to reverse.

Aurelius closes that gap by moving from an **observe, decide** loop to an **observe, forecast, simulate, decide** loop. It is a different control architecture, not another scheduler.

02 Control loop

At each control period, Aurelius reads the current fleet state and forecasts conditions across a finite planning horizon. It generates candidate sequences of coupled control actions, rolls each sequence through the predictive fleet model, rejects trajectories that violate hard constraints, and scores the rest by risk-adjusted SLA-safe goodput per dollar. Only the first action from the highest-scoring feasible sequence is returned to the existing control plane; Aurelius then replans from the next observed state.

Aurelius evaluates the controls jointly. Workload timing, placement, routing, capacity, batching, precision, and clock policy are represented as a coupled control bundle because changing one can alter the value of the others. Evaluating these controls independently can therefore miss combinations that improve SLA-safe goodput per dollar.

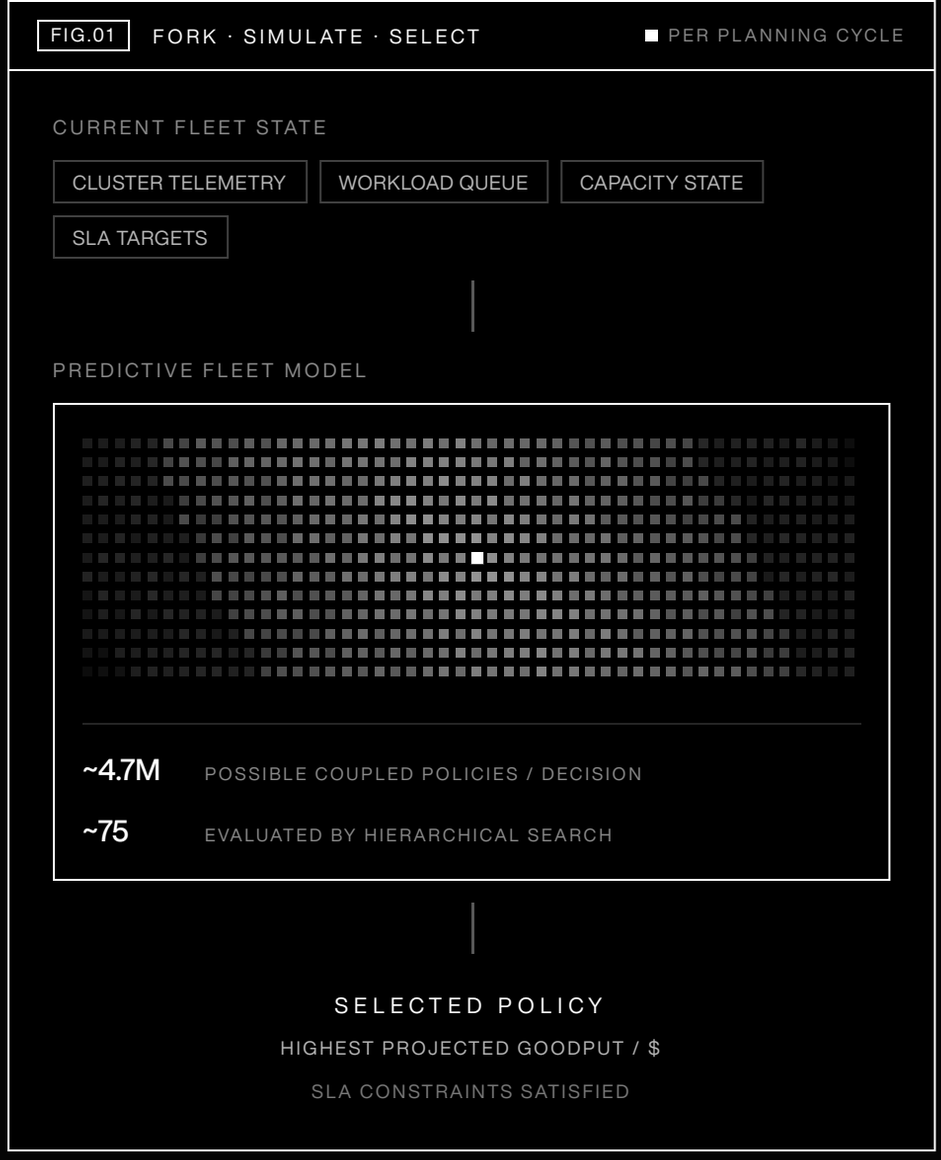

FIG.01 · ONE PLANNING CYCLE: MANY COUNTERFACTUAL TRAJECTORIES FORKED, ADVANCED UNDER CONSTRAINTS, ONE RETURNED · SCHEMATIC, NOT TO SCALE

03 Predictive fleet model

Aurelius maintains a forward model of replica state, server and rack capacity, placement and locality, queue dynamics, in-flight migrations, and operator cost. The model advances this state one control period at a time, allowing each candidate sequence to be scored against its projected downstream fleet state.

Candidate policies are evaluated on read-only simulated future states, so scoring never mutates the live cluster. Each input is labeled as measured, trace-derived, benchmark-calibrated, or modeled.

04 Candidate generation and search

Because the controls interact, candidate generation affects the quality of the result. A forecast-guided generator proposes a focused set of coupled policies for the current operating regime and always includes the predefined strong candidates. Hierarchical search then evaluates interactions across control dimensions, expands when the leading candidates are close, and stops early when the ranking is stable.

Where exhaustive enumeration is tractable, the bounded search is compared against it and search regret is measured directly.

DECISION LEVERS REPRESENTED BY AURELIUS SCORED AS COUPLED BUNDLES; EACH CHANGES THE PROJECTED ECONOMIC OUTCOME

Capacity policy · Capacity multiplier · Ordering · Admission · Batching · Routing · Prewarm · Placement · Migration · Precision · Clock / DVFS

05 Ablation: policy representation vs. search effort

Most production control loops optimize one action at a time and re-plan after execution. Aurelius instead evaluates sequences of coupled actions across multiple future control periods, then commits only the first action from the highest-scoring feasible sequence.

The ablation holds the forecast, simulator, objective, cost model, gates, replay inputs, and baseline fixed. Only two things varied: the planner's representable policy space and the procedure used to search it. Every arm receives identical replay inputs and is evaluated under identical scoring and constraint logic.

TABLE 1 · SAME UNCAPPED HARNESS, VARYING ONLY REPRESENTABLE POLICIES AND SEARCH · SLA-SAFE GOODPUT/$ RELATIVE TO THE RECONSTRUCTED BASELINE (MEAN ACROSS PJM, ERCOT, CAISO)

SEARCH STRATEGY	GP/$ VS BASELINE	SLA VS BASELINE	NOTE
Reconstructed production-class baseline	1.00×	–	reference anchor
Single lever (clock only)	does not complete	–	times out under uncapped load
Fixed set of 24 predefined policies	3.62×	~4× lower	coupling, no search
Search that fixed set (bounded)	4.85×	~4× lower	matches full enumeration of the set
Expanded coupled-policy space, same controls	8.24×	lowest, ~18× lower	8.24× configuration

Every arm is scored against the same baseline (130,538.91 / 130,178.26 / 130,000.36 gp/$ across the three markets); the coupled-search arm is the §06 configuration, quoted at 8.24×. Simulated; every completing arm also lowers the SLA-violation rate, the coupled arm the most (to ~0.002).

The predefined grid contains 24 policies over four implemented controls: batching, capacity, precision, and clock. The expanded arm constructs additional valid combinations of those same four controls. It introduces no new control, and all policies are evaluated under the same simulator, objective, cost model, and safety gates.

The ablation supports three conclusions:

- Jointly setting the four controls reaches 3.62× the reconstructed baseline and reduces SLA violations by approximately fourfold.
- Searching the same 24-policy set provides limited additional gain: bounded search reaches 4.85×, matching exhaustive evaluation of the same set at 4.84×.
- The remaining gain comes from expanding the representable policy space: the increase from 4.85× to 8.24× appears when Aurelius constructs additional valid combinations of those same four controls rather than being restricted to the original 24 policies.

The simulator, objective, cost model, constraints, and replay inputs remain fixed across every arm. In windows where complete enumeration is tractable, bounded search matches the exhaustive

optimum exactly. Transfer of the absolute magnitude to a real fleet requires validation on operator telemetry, as described in §11.

> The aggressive int4 mode is excluded from the reported result because its quality cost is not yet modeled. The reported result uses only precision settings with defensible quality assumptions.

06 Uncapped replay results

The benchmark replays the full selected windows with the per-period request cap removed, so each runs at its natural, production-scale request volume: PJM ~577k, ERCOT ~443k, CAISO ~530k, about 1.55M in total. Both policies run the identical load. Because no operator's internal scheduler is publicly available, the benchmark uses a reconstructed production-class baseline assembled from established components used in modern serving stacks: continuous (iteration-level) batching[1][2], SLA-aware ordering and admission[3], prefix-cache-aware routing[4], topology-aware placement[5], backlog-driven autoscaling[6], and warm-pool headroom. It is a reproducible approximation of a capable production stack using public mechanisms and data, not any particular operator's proprietary scheduler.

At that volume the naive SLA-aware policy does not complete: its no-batching, no-admission replay cost grows super-linearly and times out, so it is reported as a reference only. The reconstructed production-class baseline and Aurelius both complete, so it is the primary comparison.

TABLE 2 · UNCAPPED HIGH-LOAD REPLAY · AURELIUS VS PRODUCTION-CLASS BASELINE

MARKET	AURELIUS GP/$	PRODUCTION GP/$	Δ %	SLA (AUR.)	SLA (PROD.)	REQUESTS
PJM · expensive	1,041,842	130,539	+698.1%	0.0023	0.0382	576,912
ERCOT · expensive	1,064,602	130,178	+717.8%	0.0013	0.0447	442,716
CAISO · expensive	1,111,915	130,000	+755.3%	0.0018	0.0458	529,621
Average / total	–	–	+723.7%	–	–	1,549,249

gp/$ and SLA-violation rates read directly from run output. Δ% is per market (Aurelius − production) ÷ production; the reported +724% is the equal-weighted average of the three (723.7%, shown unrounded). SLA violations fall from ~3.8–4.6% to ~0.1–0.2%.

TABLE 3 · GPU-HOURS AT THE SAME UNCAPPED LOAD

MARKET	AURELIUS GPU-H	PRODUCTION GPU-H	Δ GPU-HOURS
PJM · expensive	62.2	399.9	−84.4%
ERCOT · expensive	51.0	323.7	−84.3%
CAISO · expensive	60.0	391.5	−84.7%
Total	173.1	1,115.1	−84.5%

Same served requests both arms; the reduction is (production − Aurelius) ÷ production. At the same request volume, Aurelius completes more SLA-compliant work with fewer GPU-hours and a lower SLA-violation rate.

07 Drivers of the 8.24× result

The 8.24× ratio is driven by two effects. First, the reconstructed baseline loses efficiency as the uncapped replay becomes saturated, while Aurelius re-evaluates its coupled policy at each control period. Second, the ablation in §05 shows that expanding the representable policy space produces the largest incremental gain under the same simulator, objective, cost model, and constraints.

The magnitude is load-dependent. It changes with request volume and fleet saturation and should not be extrapolated as a constant across workloads or operating conditions. The reported ratio applies only to the selected uncapped high-load windows.

The mechanism result is separate from the absolute magnitude: across the tested loads, jointly representing and evaluating coupled policies outperforms restricting the planner to the predefined policy set. Validation on operator telemetry is still required to determine the size of that effect on a real fleet.

08 Technical contribution

Aurelius combines established methods in forecasting, model-predictive control, infrastructure simulation, SLO-aware scheduling, and unified control planes around a multi-period fleet-control objective.

The ablation isolates the contribution of representable policy space and coupled evaluation while holding the simulator and scoring environment fixed. The same mechanism can be tested on any fleet with sufficiently detailed historical scheduler metadata.

09 Safety and deployment

Aurelius is **read-only and metadata-only**. It reads only the metadata a scheduler already exposes, job timing, resource requests, constraints, and never payloads, model weights, or model outputs. Candidate scoring runs on simulated futures, so it writes nothing to the live cluster. Candidates that violate hard constraints are discarded before scoring and cannot be returned to the control plane. When confidence is low the controller falls back deterministically to the stack's default.

Validation proceeds in three stages: historical replay, recommendation-only shadow mode, and optional actuation on explicitly scoped workloads. Aurelius does not directly write fleet state; it returns a constrained recommendation to the existing control plane, which remains authoritative and executes any approved action.

10 Limitations

- Every number here is deterministic simulated replay of public traces: evidence from simulation, not a production estimate, guarantee, or deployment.
- The +724% figure is load-dependent. Part of the gap is a fixed-policy scheduler degrading under heavy uncapped load while Aurelius adapts, so it is reported only with its uncapped, full-window scope and never as a fleet constant.
- Magnitudes are simulator-inferred and depend on the serving model's precision and batching bands. The mechanism attribution is more defensible than the absolute percentage, which remains simulator-dependent.

- Aggressive low-precision is excluded from the reported result until its quality cost is modeled.
- Simulator fidelity must still be tested against real operator telemetry, which is the only way to isolate model error, and the largest gains appear during expensive-power windows.
- Some controls are modeled but not yet active production levers, and several constraints are represented but not yet forecast inputs.

11 Validation on operator telemetry

The next validation step is replay on operator telemetry. The public baseline should next be replaced with an operator's recorded decisions and fleet state. Given read-only historical scheduler metadata, Aurelius would replay that fleet's recorded behavior, simulate the coupled counterfactuals, and estimate the change in SLA-safe goodput per dollar under the same arrivals, infrastructure state, and operator constraints, with no production changes and no payload access.

The evaluation begins with a baseline-reconstruction check. The calibrated simulator must reproduce recorded queue, execution, resource-use, and SLA outcomes within agreed tolerances. Only after that check passes are counterfactual policies evaluated on held-out historical windows. If either the fidelity criteria or the pre-agreed performance criterion fails, the evaluation stops without any production change. A successful replay would support recommendation-only shadow mode. A failed replay would end the evaluation without affecting production.

· Reproducibility notes & references

BENCHMARK CONFIGURATION

- **Primary metric.** SLA-safe goodput per infrastructure dollar. The numerator is SLA-compliant served tokens, tokens served within their latency or deadline target; work served outside that target is excluded, so goodput bought by breaking SLA does not count. The denominator, modeled infrastructure cost, is the sum of three terms: GPU cost (active GPU-hours, including warm-held replicas, at a fixed hardware cost basis), energy cost (GPU energy at the prevailing wholesale electricity price), and network cost (including migration). The GPU cost basis is hardware only, so energy is counted once, at the market price. This is a modeled benchmark cost, not full operator TCO.
- **Windows and load.** The expensive-power windows of PJM, ERCOT, and CAISO, replayed uncapped at full volume (~577k / ~443k / ~530k requests, ~1.55M total), three control-period decisions per window.
- **Configuration.** Deterministic replay under a fixed seed; every policy is scored on the same windows under the same simulator, cost model, and constraint gates. Forecasts use only history up to each decision, and per-request execution time is replayed rather than predicted; no forecast uses information recorded after the decision timestamp. The baseline's goodput per dollar reproduces exactly across deterministic runs (130,538.91 / 130,178.26 / 130,000.36 gp/$). At each decision the hierarchical search evaluates roughly 75 coupled policies out of a representable space of about 4.7M.

The reconstructed production-class baseline (§06) is assembled from established production serving and cluster-scheduling techniques ([1]–[6]), cited for the realism of the baseline mechanisms, not for any Aurelius result. The public workload and electricity-price data are cited at [7] and [8].

[1] Yu et al. “Orca: A Distributed Serving System for Transformer-Based Generative Models.” OSDI 2022. Iteration-level (continuous) batching.

[2] Kwon et al. “Efficient Memory Management for Large Language Model Serving with PagedAttention” (vLLM). SOSP 2023. Continuous batching and KV-cache management.

[3] Gujarati et al. “Serving DNNs like Clockwork: Performance Predictability from the Bottom Up.” OSDI 2020. SLO-aware scheduling and admission.

[4] Zheng et al. “SGLang: Efficient Execution of Structured Language Model Programs” (RadixAttention). 2024. Prefix / KV-cache-aware reuse.

[5] SchedMD. “Slurm Workload Manager, Topology Guide.” Topology-aware placement in production cluster schedulers.

[6] Rzadca et al. “Autopilot: Workload Autoscaling at Google.” EuroSys 2020. Utilization-headroom autoscaling at production scale.

[7] Workload, public serving traces: Wang et al. “BurstGPT: A Real-world Workload Dataset to Optimize LLM Serving Systems” (2024), for arrivals and request characteristics; and the Mooncake trace, for prefix / KV-cache reuse.

[8] Electricity, public wholesale prices: CAISO OASIS, PJM Data Miner, and ERCOT, day-ahead and real-time locational marginal prices for the selected PJM, ERCOT, and CAISO windows.